

Технологии программирования

Сборка мусора

Введение

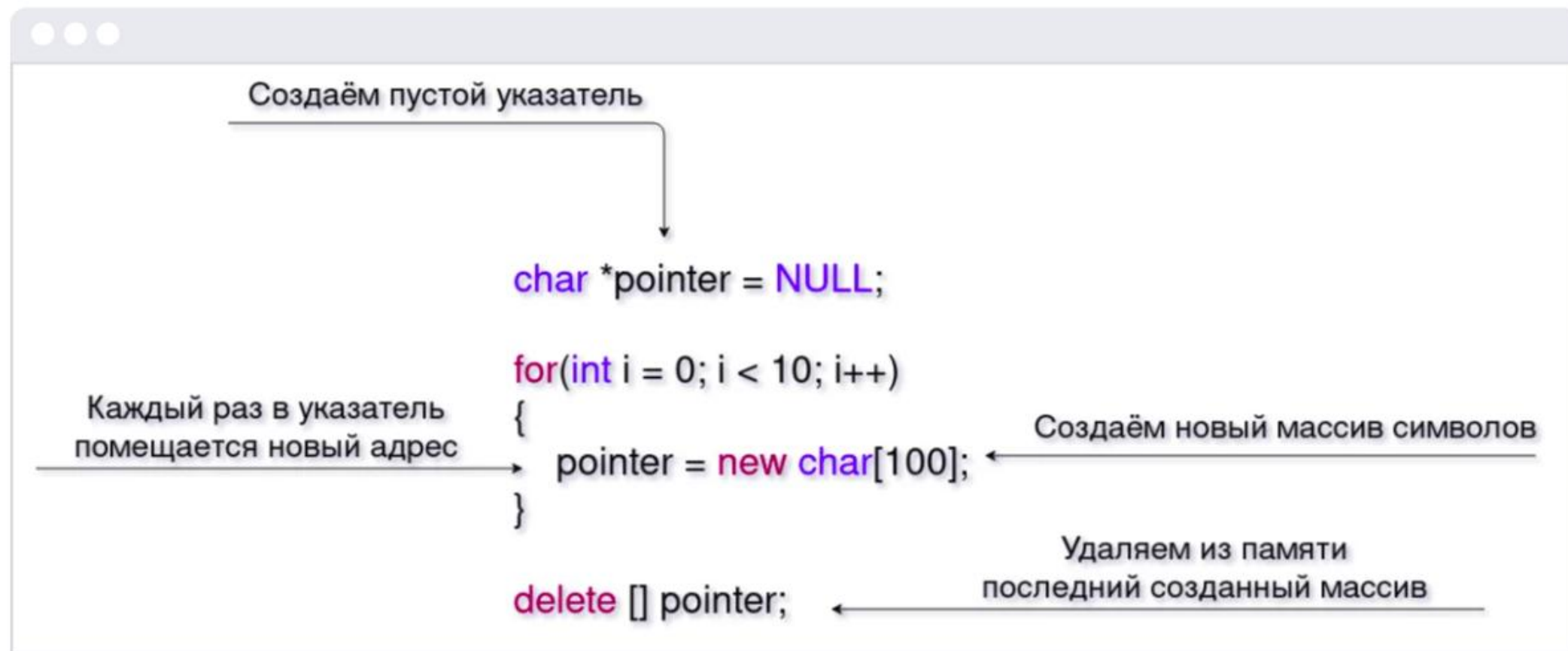
Что такое сборка мусора?

Сборка мусора — это процесс восстановления заполненной памяти среды выполнения путем уничтожения неиспользуемых объектов.

Утечка памяти

В таких языках, как С и С++, программист отвечает как за создание, так и за уничтожение объектов. Иногда программист может забыть уничтожить бесполезные объекты, и выделенная им память не освобождается. Расходуется все больше и больше системной памяти, и в конечном итоге она больше не выделяется. Такие приложения страдают от “утечек памяти”.

Однако иногда случаются ошибки, которые приводят к утечкам памяти. Вот одна из них:



На примере в цикле десять раз создаётся новый массив, а его адрес записывается в указатель. Адреса старых массивов при этом удаляются. Поэтому дальше оператор `delete[]` удаляет только последний созданный массив. Остальные останутся в памяти до тех пор, пока не будет закрыта программа.

Утечка памяти

После определенного момента памяти уже не хватает для создания новых объектов, и программа внештатно завершается из-за `OutOfMemoryErrors`.

В C++ для сборки мусора можно воспользоваться оператором `delete`, а в C — методом `free()`. В managed-языках сборка мусора происходит автоматически в течение всего времени работы программы. Это устраняет необходимость выделения памяти и, следовательно, позволяет избежать утечек.

Утечка памяти

Не допускать ситуаций с утечками памяти вроде бы и не трудно — нужно всего лишь "класть на место всё что взяли", но на практике это очень сильно осложняется хитростью архитектуры, нелинейным порядком выполнения операторов, например, из-за применения исключений, а также человеческим фактором.

Человеку присуще делать ошибки, что-то забывать и пропускать. И по отношению к памяти такие вещи крайне опасны, губительны, а иногда и непростительны.

При этом не менее важна не только утечка памяти, но и доступ уже к высвобожденным частям. Нередки ситуации, когда память освобождена, а указатель на нее остался и разработчик по этому указателю обращается, в результате чего случаются ужасные вещи. Например ошибка по типу Segmentation fault.

Утечка памяти

Учитывая еще то, что вы должны думать о бизнес логике в приложении получается двойная нагрузка на разработчика.

Разумеется, логичным желанием было бы делегировать эту рутинную работу кому-то.

Существует даже специальные программы-анализаторы, задачей которых является как раз поиск проблемных мест и утечек памяти, то т.к. мы используем высокоуровневые языки с автоматическим управлением памятью, об анализаторах используемых в С/С++ сегодня мы говорить не будем.

Автоматическое управление памятью

Как вы знаете, Java код транслируется в байткод, который уже в свою очередь выполняется JVM.

Таким образом, JVM играет ключевую роль в работе приложения и предоставляет разработчикам преимущества, среди которых есть одно настолько значимое, что не поговорить об этом было бы преступлением.

Это автоматическое управление памятью.

Автоматическое управление памятью

Важно понимать, что существуют такие механизмы как unsafe-код, которые дают возможность разработчикам вручную управлять памятью среды выполнения, но, не всегда доступны в конкретных реализациях языка.

Несмотря на то что в C# и Java такие механизмы есть, их используют в очень специфических сценариях и в таком случае нужна ручная настройка сборки мусора

Автоматическое управление памятью

Инструмент используемый в managed-языках для высвобождения неиспользуемой памяти называется **Garbage Collector** (сборщик мусора)

У GC всего две задачи

- Обнаружение мусора
- Отчистка мусора

Garbage Collection

Мусор - это структура данных(объект) в памяти, к которому из программного кода больше невозможно получить доступ.

Как если бы вы потеряли ссылку на то что вам выделилось в heap'e после вызова malloc(...)

Garbage Collection

Существует несколько реализаций GC, работающих по различным алгоритмам, каждый из которых по своему решает проблему отслеживания и уничтожения уже ненужных объектов. Какие-то реализации лучше работают на больших размерах heap-а, какие-то лучше работают на средних размерах и меньше.

При этом по спецификации нет никаких правил для реализации GC, кроме сбора объектов, которые гарантированно более не могут быть использованы.

Garbage Collection

Как определить, что объект не нужен, что объект не используется и это мусор?

Существует несколько подходов для поиска мусора:

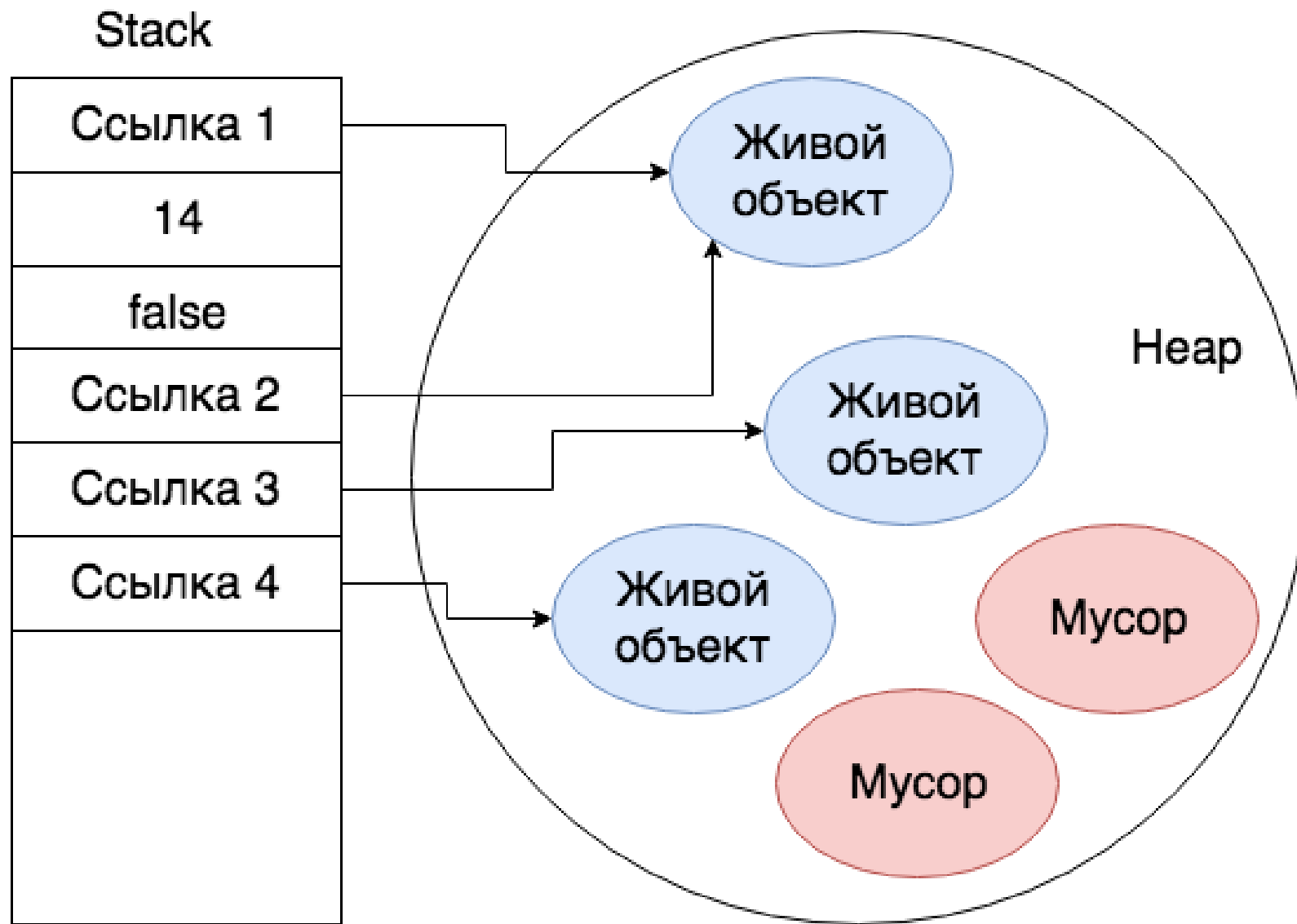
- Reference counting
- Tracing

Reference Counting

Данный подход основан на подсчете ссылок, о чем можно догадаться из названия.

Суть подхода состоит в том, что каждый объект имеет некоторый счетчик. Этот счетчик хранит информацию о том, сколько ссылок указывает на объект. Когда какая-либо ссылка уничтожается, то и значение счетчика уменьшается.

Если значение счетчика равно нулю - объект можно считать мусором и память, которую он занимает, можно очищать.



Reference Counting

Используется во многих языках программирования, например, в Python.

У этого подхода есть много плюсов: он прост, не требует долгих пауз для сборки мусора.

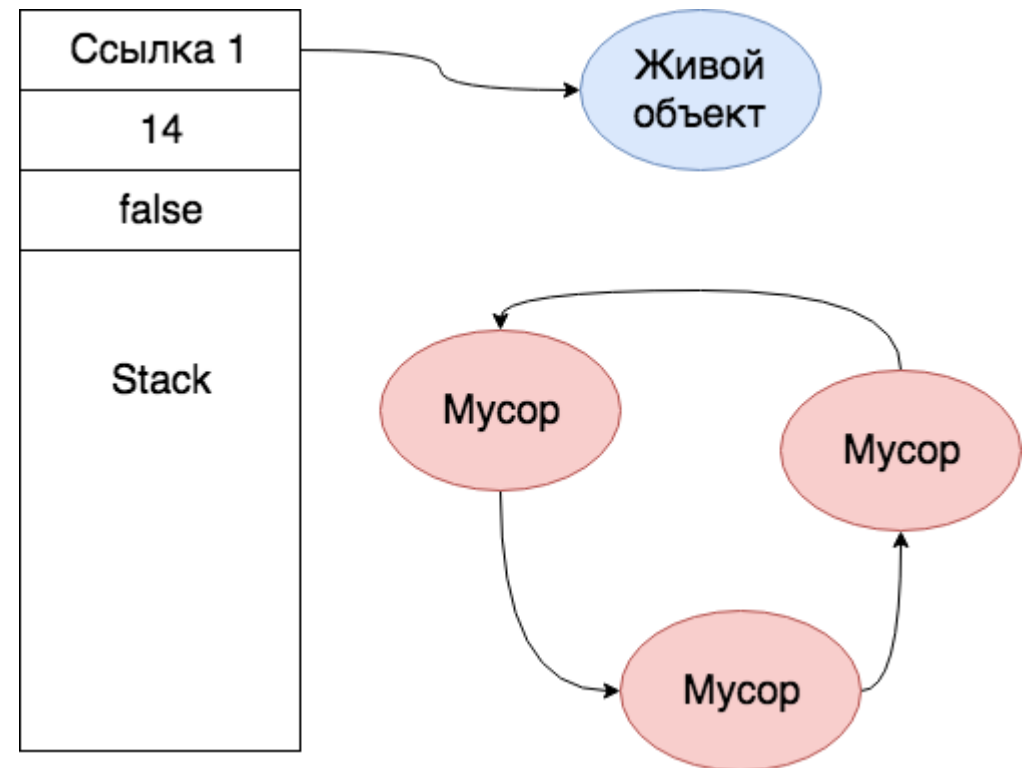
Однако, есть и несколько существенных минусов:

- Плохо сочетается с многопоточностью
- Сложно выявлять циклические зависимости, что требует большой ответственности для реализации точности счетчика.
- Влияет на производительность – каждый раз, когда мы что-то читаем, записываем ссылку на объект в локальную переменную, нам нужно увеличивать счетчик.

Reference Counting

Циклические зависимости - когда два объекта указывают друг на друга, но ни один живой объект на них не ссылается. Это приводит к утечкам памяти.

Каждый из объектов на рисунке имеет по одной ссылке на себя, однако только один является по-настоящему живым.



Reference Counting

Благодаря своим минусам данный подход не используется и вытеснен более гибким подходом, под названием **Tracing**

Tracing

Этот подход вводит новое понятие - GC Root

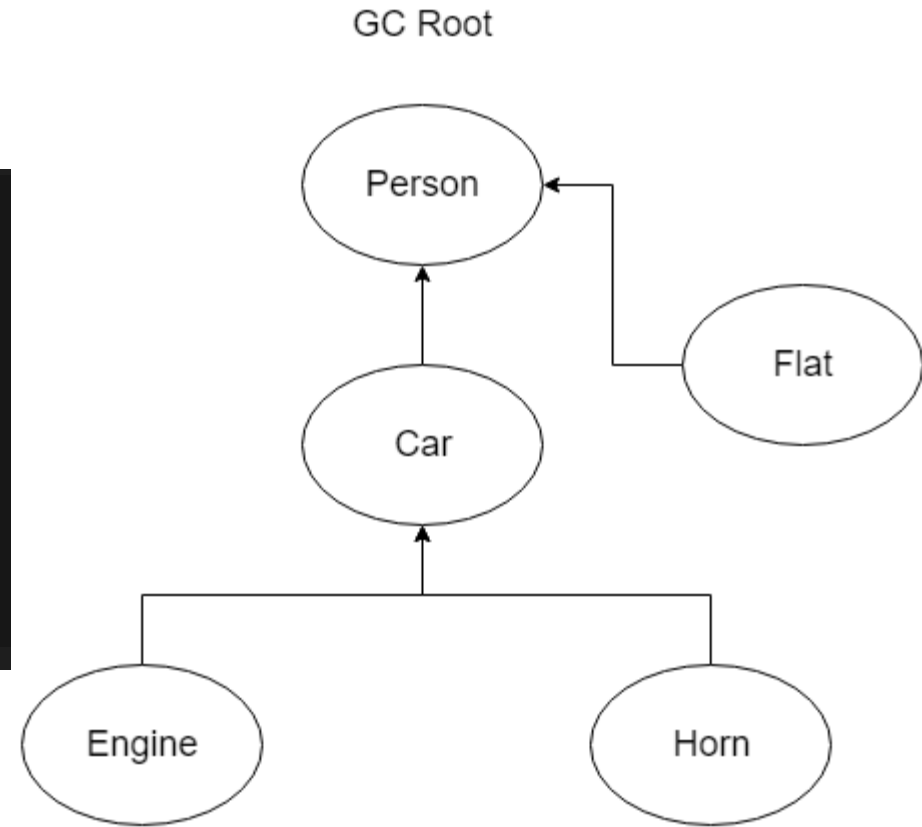
Главную идею подхода можно сформулировать так:

Живые объекты - это те, до которых мы можем добраться от корня (GC Root), в то время как все остальные являются мусором. Все что доступно с живого объекта - также живое.

Рассмотрим на примере...

Tracing

```
Person p = new Person();  
p.setFlat(new Flat());  
p.setCar(new Car());  
p.getCar().setEngine(new Engine());  
p.getCar().setHorn(new Horn());
```

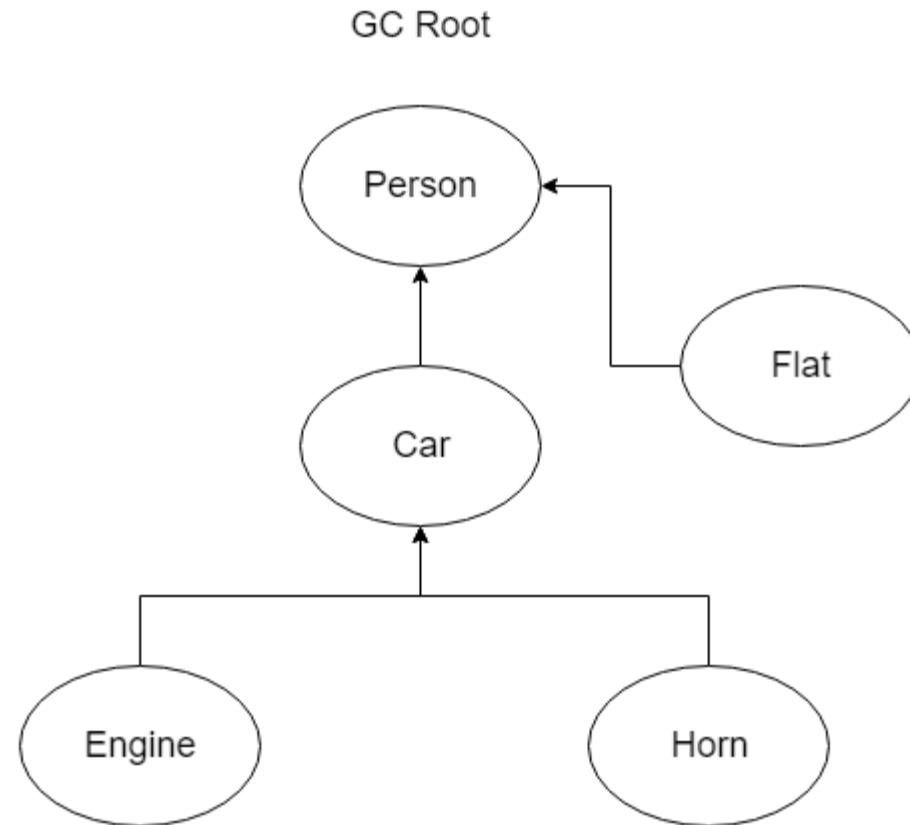


Tracing

Так вот Person - это и есть тот самый корень, якорь.

Т.е это наивысшая точка графа связанных объектов.

Так как Person у нас является живым объектом, то считается, что все объекты, до которых мы можем добраться из Person - также живые.

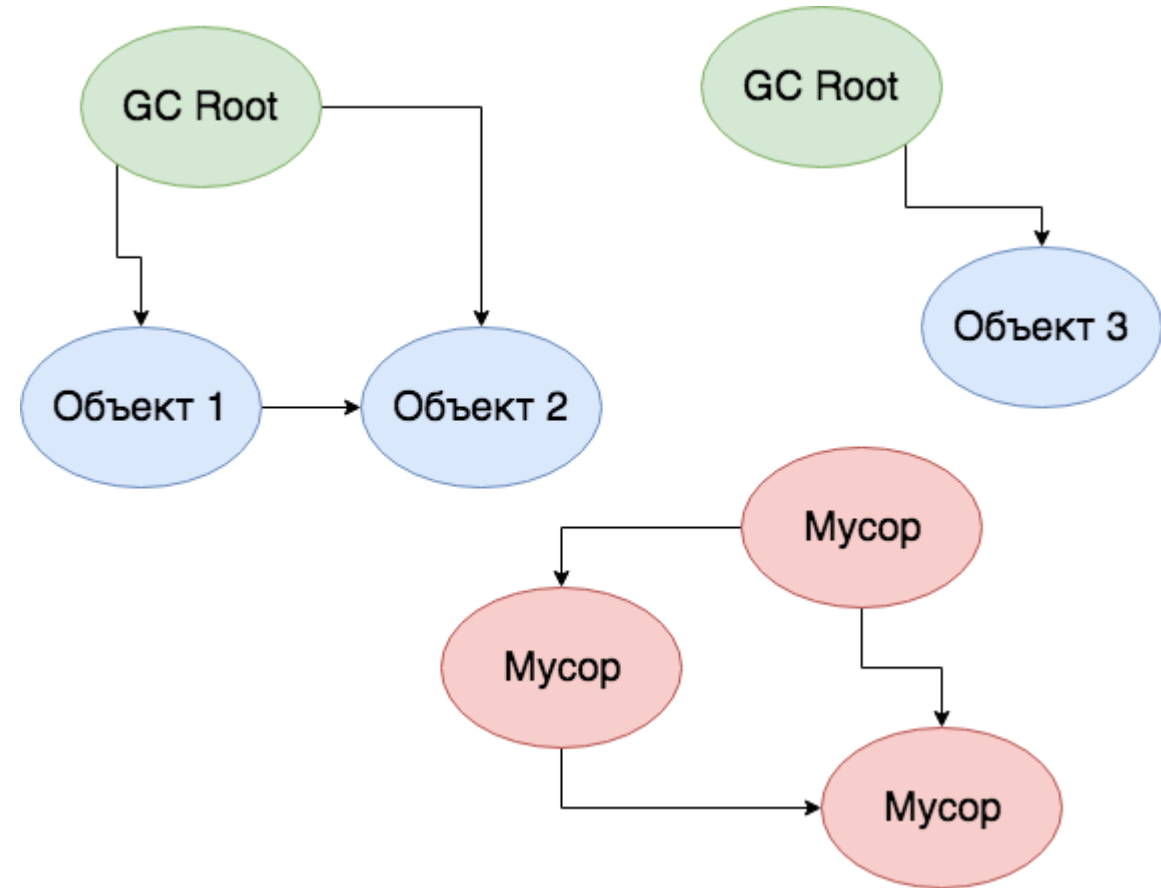


Tracing

Проговорим еще раз смысл разбираемого подхода:

Если мы представим все объекты и ссылки между ними как дерево, то нам нужно пройти от корневых узлов по всем узлам.

При этом узлы, до которых мы сможем добраться - не мусор, все остальные - мусор.



Tracing

Видно, что одна из проблем reference counting, с циклическими зависимостями, сейчас решается сама собой. Все просто: не можем добраться до объекта - значит объект мусор.

Логичным образом всплывает вопрос: какие бывают GC Root? Как мы уже поняли из примера с Person, что **локальные переменные** являются **GC Root**.

Score жизни GC Root

Вернёмся к примеру и предположим, что код лежит внутри некоторого метода:

После сборки мусора
объект по ссылке **p**
и все связанные с ним
другие объекты
не выживут.

Но, почему?

```
void foo() {  
    Person p = new Person();  
    p.setFlat(new Flat());  
    p.setCar(new Car());  
    p.getCar().setEngine(new Engine());  
    p.getCar().setHorn(new Horn());  
  
    System.gc();  
}
```


Scope жизни GC Root

Дело в том, что компилятор вычисляет для всех переменных live ranges - это все, что находится на любых путях исполнения от определения переменной до последнего использования (или использований, они могут быть разные на разных путях).

Другими словами, компилятор всегда знает: жива сейчас переменная или нет, могут ее потенциально в будущем еще использовать или нет.

GC тоже ориентируется на это знание от компилятора. Если переменная вне своего live range, то значит она уже не будет корнем.

GC Root Types

что еще может быть корневой точкой?

В Java 8 корневой точкой могут быть:

- Локальные переменные и параметры методов
- Потоки Java
- Статические переменные
- Ссылки из JNI*

JNI - Java Native Interface — стандартный механизм для запуска кода под управлением виртуальной машины Java, который написан на unmanaged-языках и скомпонован в виде динамических библиотек

GC Roots

```
class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Из этого следует, что даже самое простое java приложение имеет следующие GC Root:

Локальные переменные внутри main метода.

Статические переменные класса, содержащего main метод.

Параметры main метода (**args**)

Поток, который выполняет main метод.

Очистка памяти

С задачей обнаружения мусора разобрались.

Теперь поговорим о том как происходит очистка.

Очистка памяти процесс довольно сложный, поэтому было также разработано несколько алгоритмов, выполняющих эту задачу.

Рассмотрим какие алгоритмы очистки существуют.

Алгоритмы отчистки памяти

- Копирующая сборка
- Отслеживание и отчистка
- Отслеживание и отчистка с дефрагментацией
- Разделение на поколения

Копирующая сборка мусора

Память условно делится на две области: **from-space** и **to-space**.

Все объекты создаются в области **from-space**, по мере заполнения этой области запускается очистка мусора. Приложение полностью останавливается - происходит так называемый **stop-the-world** - в момент начала очистки, после чего все "живые" объекты в **from-space** копируются в **to-space**.

После того, как все "живые" объекты скопированы происходит полная очистка **from-space** и области меняются местами.

Stop the world

Простые сборщики мусора с остановкой мира полностью останавливают выполнение программы для запуска цикла сбора, тем самым гарантируя, что новые объекты не будут выделены и объекты не станут внезапно недоступными во время работы сборщика.

Stop the world

Благодаря stop the world паузе нам:

- Проще определять достижимость объектов, так как граф объектов в этот момент заморожен.
- Проще перемещать объекты в куче, так как можно не бояться, что мы что-то сломаем и мы можем ненадолго перевести кучу в некорректное состояние для наших нужд.

Для тех задач, где пауза критична существует такое понятие как инкрементальная сборка. Там мы делаем большое количество кратковременных пауз. Это выражается в большей нагрузке на приложение.

Mark-and-Sweep

Данный алгоритм называется Mark-and-Sweep: "отслеживание и очистка".

Алгоритм очень похож на предыдущий, но с некоторыми улучшениями.

Объекты аллоцируются в памяти и в какой-то момент запускается очистка мусора. Приложение полностью останавливается - здесь все также, как и в предыдущем случае, без остановки никуда.

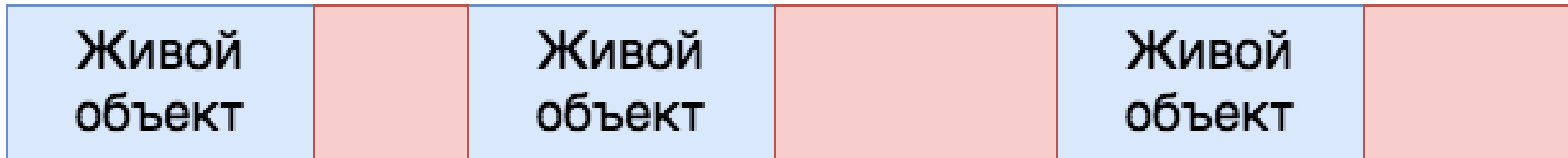
После остановки мы проходим по всем объектам и помечаем(mark) все "живые" объекты, после чего делаем sweep - чистим и снимаем все пометки с "живых" объектов.

Mark-and-Sweep

Главным минусом подхода является то, что память становится фрагментированной. Так как получаются целые куски свободной памяти после sweep.

Также при большом количестве "живых" объектов работа алгоритма становится гораздо менее эффективной.

Проиллюстрируем это, красным выделена очищенная область:



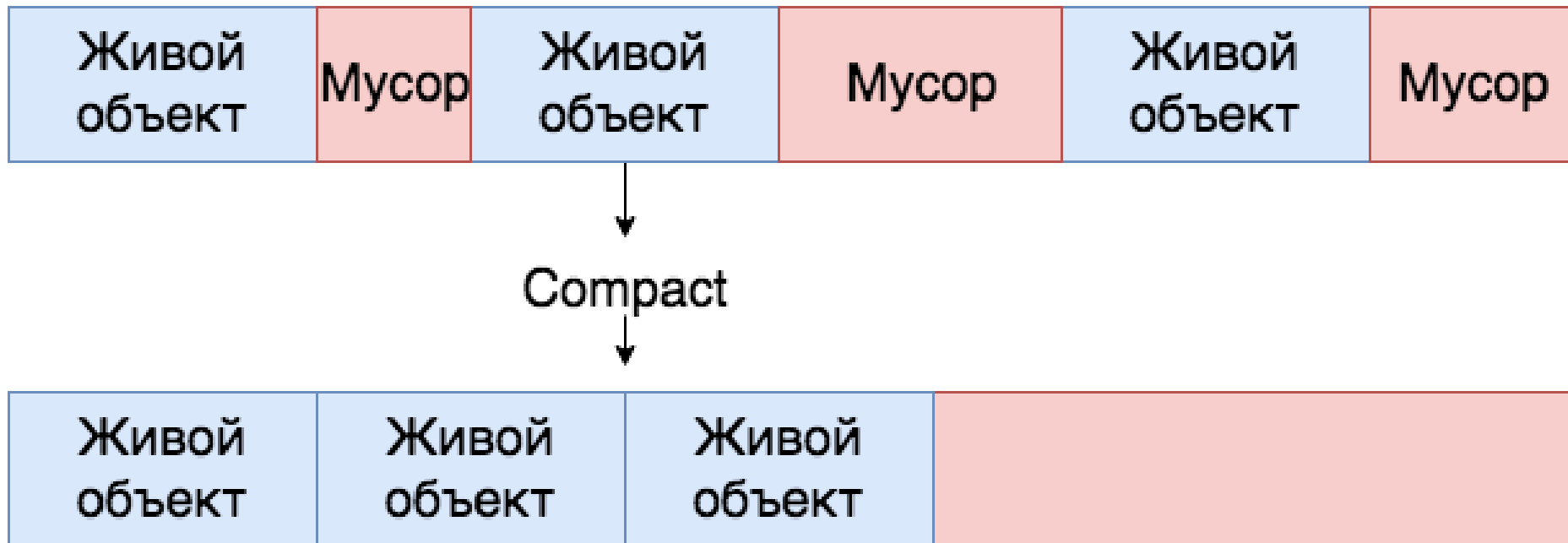
Mark-and-Sweep Compact

В отличии от простого Mark-and-sweep мы ищем "мертвые" объекты, помечаем их для переноса и только после этого останавливаем приложение для очистки памяти.

Так как с "мертвыми" объектами наше приложение уже не работает мы можем искать их параллельно работе приложения. Это очень эффективно, так как мы теперь не тратим время паузы на поиск, как в предыдущих алгоритмах.

Mark-and-Sweep Compact

После завершения процедуры удаления происходит compact - мы дефрагментируем память. Объекты "сдвигаются" на более близкие адреса.



Mark-and-Sweep Compact

Плюсы:

- Нет фрагментации памяти.
- Эффективная работа при большом количестве "живых" объектов.

Минусы:

- Плохо работает при большом количестве "мертвых" объектов.
- Compact - дорогостоящая операция, занимающая много времени.

Сборка мусора на поколениях в Java

Сразу договоримся, что будем рассматривать то как это работает в HotSpot JVM (Виртуальная машина используемая в Oracle JDK по умолчанию). В других имплементациях описанный далее алгоритм может работать по другому =)

Сборка мусора на поколениях в Java

Исходя из анализа уже работающих систем было выделено две закономерности.

Первая из них гласит, что большинство объектов в программе либо живут очень долго, либо очень недолго. Более того, количество объектов живущих долго крайне невелико.

Вторая закономерность гласит, что существует очень мало связей между "старыми" объектами, которые уже существовали давно, и "новыми", только что или недавно созданными, объектами.

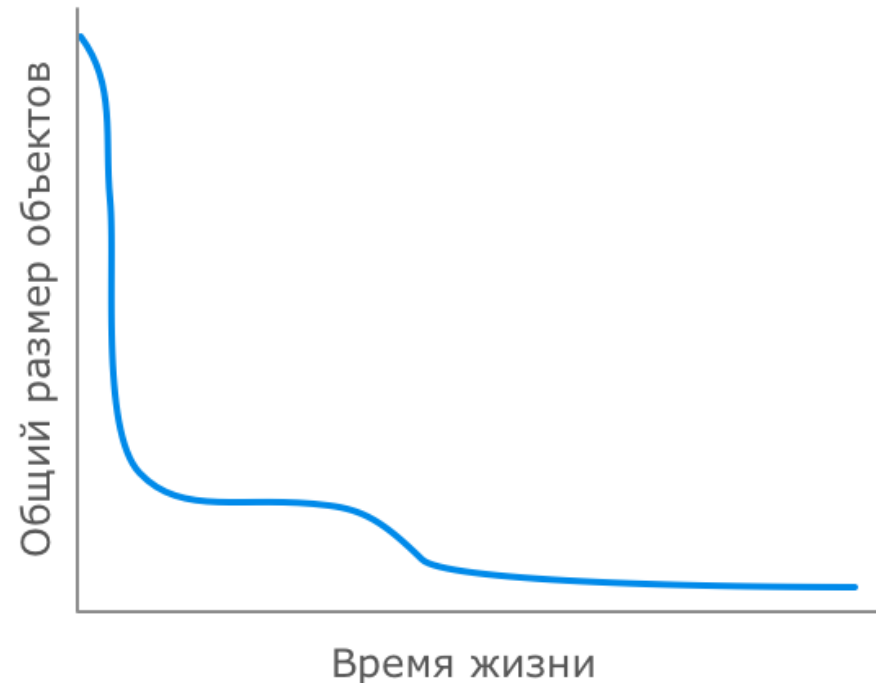
Эти две закономерности еще иногда называют "слабой гипотезой о поколениях".

Сборка мусора на поколениях в Java

В общем виде, гипотеза о поколениях гласит, что вероятность смерти как функция от возраста снижается очень быстро. Ее приложение к сборке мусора в частности означает, что подавляющее большинство объектов живут крайне недолго. По людским меркам, большинство даже в детский сад не пойдут. Также это означает, что чем дольше прожил объект, тем выше вероятность того, что он будет жить и дальше.

Сборка мусора на поколениях в Java

Большинство приложений имеют распределение времен жизни объектов, схематично описываемое примерно такой кривой:



Сборка мусора на поколениях в Java

Подавляющее большинство объектов создаются на очень короткое время, они становятся ненужными практически сразу после их первого использования. Итераторы, локальные переменные методов, результаты боксинга и прочие временные объекты, которые зачастую создаются неявно, попадают именно в эту категорию, образуя пик в самом начале графика.

Сборка мусора на поколениях в Java

Далее идут объекты, создаваемые для выполнения более-менее долгих вычислений. Их жизнь чуть разнообразнее — они обычно гуляют по различным методам, трансформируясь и обогащаясь в процессе, но после этого становятся ненужными и превращаются в мусор. Благодаря таким объектам возникает небольшой бугорок на графике следом за пиком временных объектов.

Сборка мусора на поколениях в Java

И, наконец, объекты-старожилы, переживающие почти всех — это постоянные данные программы, загружаемые часто в самом начале и проживающие долгую и счастливую жизнь до остановки приложения. (Различные Singleton'ы, ThreadPool'ы и т.д.)

Сборка мусора на поколениях в Java

Вот тут и возникает идея разделения объектов на *младшее поколение (young generation)* и *старшее поколение (old generation)*. В соответствии с этим разделением и процессы сборки мусора разделяются на *малую сборку (minor GC)*, затрагивающую только младшее поколение, и *полную сборку (full GC)*, которая может затрагивать оба поколения. Малые сборки выполняются достаточно часто и удаляют основную часть мертвых объектов. Полные сборки выполняются тогда, когда текущий объем выделенной программе памяти близок к исчерпанию и малой сборкой уже не обойтись.

Характеристики сборщиков мусора на поколениях

Интуитивно понятно, что желательно иметь сборщик мусора, который как можно быстрее избавлялся бы от ненужных объектов.

Хочется делать сборку мусора эффективно и быстро.

Поэтому прежде чем двигаться дальше, давайте разберемся с критериями, используемыми при оценке сборщиков.

Характеристики сборщиков мусора на поколениях

Традиционно, при определении эффективности работы сборщика мусора учитываются следующие факторы:

- Максимальная задержка — максимальное время STW, на которое сборщик приостанавливает выполнение программы для выполнения одной сборки.
- Пропускная способность — отношение общего времени работы программы к общему времени простоя, вызванного сборкой мусора, на длительном промежутке времени.
- Потребляемые ресурсы — объем ресурсов процессора и/или дополнительной памяти, потребляемых сборщиком.

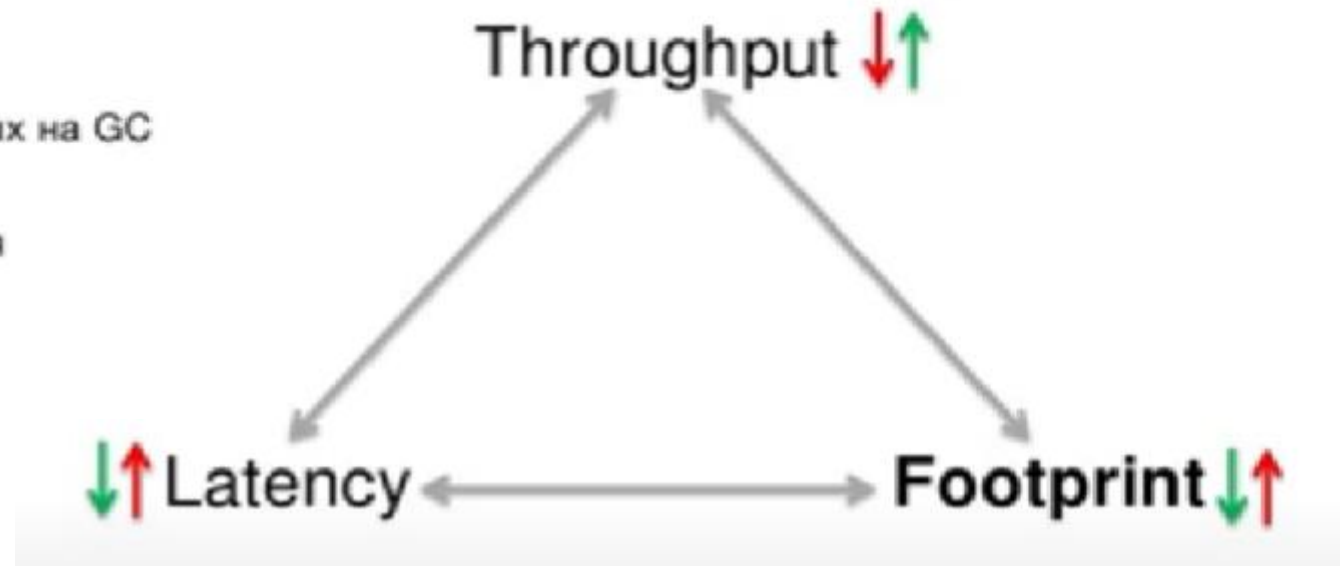
Характеристики сборщиков мусора на поколениях

Понятно, что достичь всех трёх показателей реально, поэтому при выборе конкретной имплементации алгоритмы сборки нужно сосредоточиться на двух конкретных характеристиках



Характеристики сборщиков мусора на поколениях

- 3 характеристики
 - **Throughput**
 - Объем вычислительных ресурсов, затрачиваемых на GC
 - **Предсказуемость**
 - На какое время прерывается работа приложения
 - **Footprint**
 - Объем используемой памяти



Поколения объектов в Java

Для оптимизации сборки мусора память кучи дополнительно разделена на четыре области. В эти области объекты помещаются в зависимости от их возраста (как долго они используются в приложении).

1.Young Generation (молодое поколение). Здесь создаются новые объекты. Область young generation разделена на три части раздела: Eden (Эдем), S0 и S1 (Survivor Space — область для выживших).

2.Old Generation (старое поколение). Здесь хранятся давно живущие объекты.

YOUNG GENERATION

SURVIVOR SPACE

OLD GENERATION

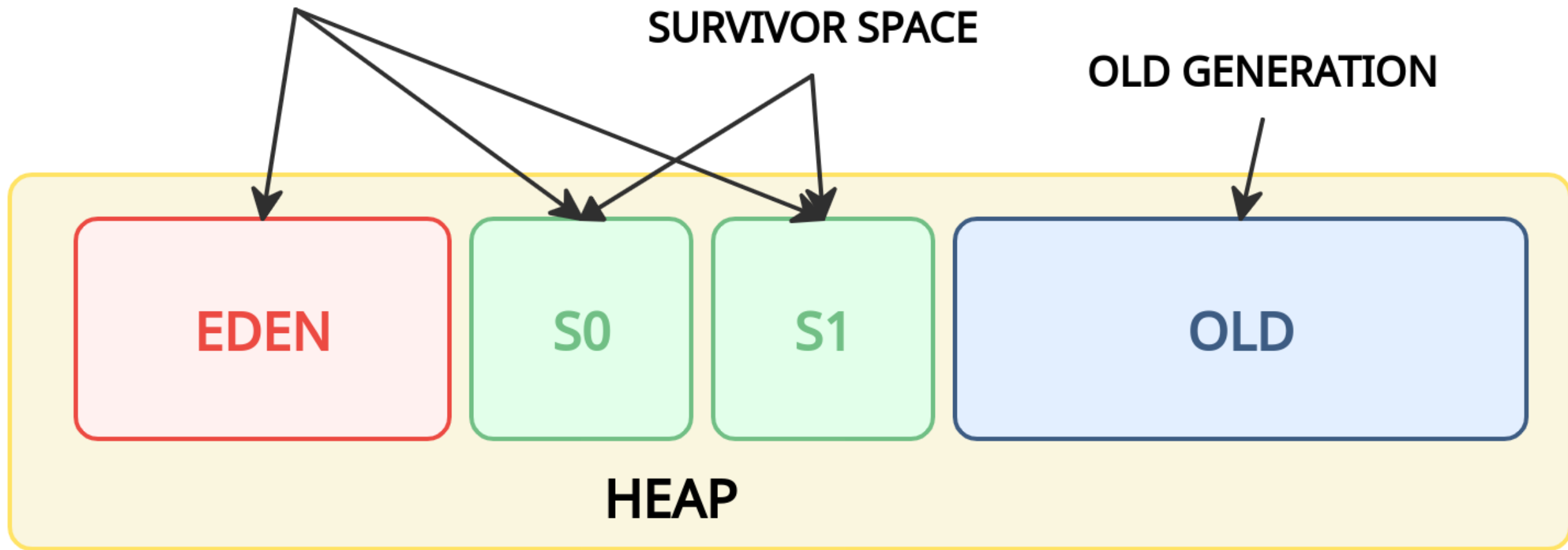
EDEN

S0

S1

OLD

HEAP



JVM Heap Memory

Эдем (Eden) — это область динамической памяти, в которой изначально создаются объекты. Многие объекты никогда не покидают этой области памяти, так как быстро становятся мусором.

Когда мы пишем что-то в виде **new Object()** мы создаем объект именно в Eden.

Относится к young generation.

JVM Heap Memory

Область уцелевших (Survivor) — как правило, в памяти присутствует две области уцелевших. Или же можно считать, что область уцелевших обычно делится пополам. Именно в нее попадают объекты, пережившие "изгнание из Эдема" (отсюда и ее название). Иногда два этих пространства называются From Space и To Space. Одна из этих областей всегда пуста, если только не происходит процесс сбора.

Из From Space объекты либо удаляются GC, либо перекочевывают в To Space - последнее место перед тем, как стать совсем старыми и перейти в Tenured.

Относится к young generation.

JVM Heap Memory

Хранилище (Tenured) — это область (также называемая "старым поколением"), где оказываются уцелевшие объекты, которые признаются "достаточно старыми" (таким образом, они покидают область Survivor).

Хранилище не очищается в ходе молодой сборки (об этом пойдет речь дальше).

Относится к old generation.

JVM Heap Memory

До Java 8 существовал специальный раздел: PermGen — здесь выделялось место для внутренних структур, например для определений классов. Строго говоря, PermGen не входило в состав динамической памяти, обычные объекты сюда никогда не попадали.

Тут хранились метаданные, классы, интернированные строки, и т.д - это была специальная область памяти у JVM.

Так как достаточно трудно понять необходимый размер этой области, то ранее, до Java 8, можно было часто наблюдать ошибку `java.lang.OutOfMemoryError`.

JVM Heap Memory

Поэтому, начиная с Java 8, было принято вообще убрать эту область и вся информация, которая там хранилась либо переносится в heap, например интернированные строки, либо выносятся в область metaspace, в native memory.

Максимальный Metaspace по умолчанию не ограничен ничем кроме предела объёма нативной памяти. Но его можно по желанию ограничить.

Из этих двух закономерностей следует логичный вывод - сборка мусора должна чаще происходить над "новыми" объектами.

Соответственно поэтому существует несколько типов сборок: minor, major и full.

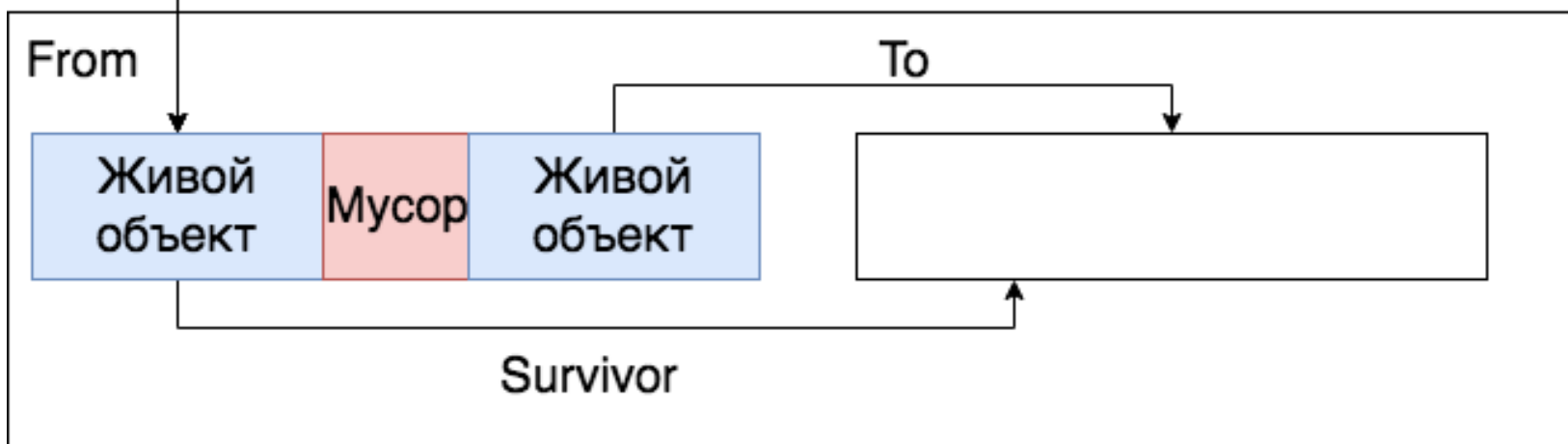
Minor сборка

В ходе minor сборки система пытается очистить только области с молодыми объектами — Eden и Survivor.

Этот процесс довольно прост.

- Все "живые" молодые объекты, найденные на этапе отслеживания, перемещаются в следующие места:
 - Объекты, которые уже достаточно стары, которые пережили достаточное количество предыдущих циклов сборки мусора, попадают из Survivor области To Space в Tenured.
 - Все остальные молодые "живые" объекты отправляются в пустую область уцелевших Survivor.
- После этого Eden и только что очищенная область в Survivor могут быть перезаписаны и переиспользованы, поскольку в них больше нет ничего, кроме мусора.

Eden



Tenured



Minor сборка

Minor сборка начинается после того, как Eden оказывается целиком заполнен.

Обратите внимание на то, что на этапе отслеживания требуется обойти весь граф живых объектов. Это означает, что если у молодого объекта есть ссылка на объект из Tenured, то ссылки, удерживаемые объектом из Tenured, также должны быть просмотрены и отслежены.

Сборки проходят часто, быстро и уничтожает кучу мусора, так как происходят на сравнительно небольшом участке памяти который скорее всего содержит много мусора.

Major сборка

В ходе major сборки система пытается очистить области с старым поколением - old generation.

Из-за того, что minor и major сборки тесно связаны, то нет смысла разбирать её отдельно, поэтому сразу перейдём к full сборке.

Full сборка

Если молодая сборка не может перевести объект в хранилище (недостаточно пространства), то запускается full.

Full сборка очищает обе области - и старое поколение, и новое.

В зависимости от того, какой механизм сборки применяется при работе со старым поколением, может потребоваться перемещать объекты в старом поколении. Это позволяет гарантировать, что в старом поколении хватает места, чтобы при необходимости выделить крупный объект.

Сборки происходят не часто, но когда происходит, занимают много времени.

Сборщики мусора, умеющие работать с такой моделью называются Generational Garbage Collection, т.е учитывающие поколения.

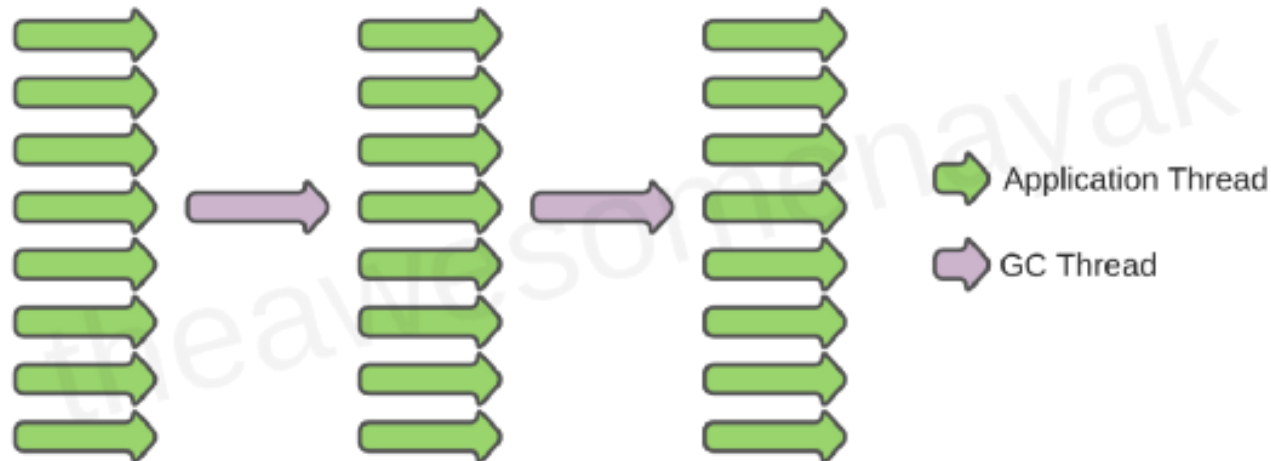
Реализации GC

В HotSpot VM реализовано несколько сборщиков мусора основанных на идее Generational Garbage Collection:

- **Serial GC**
- **Parallel GC**
- **CMS GC**
- **G1 GC**
- *Epsilon GC (Java 11+)*
- *Shenandoah GC (Java 15+)*
- *ZGC GC (Java 15+)*

Serial GC

Это самая простая реализация GC. Она предназначена для небольших приложений, работающих в однопоточных средах. Все события сборки мусора выполняются последовательно в одном потоке. Уплотнение выполняется после каждой сборки мусора.



Serial GC

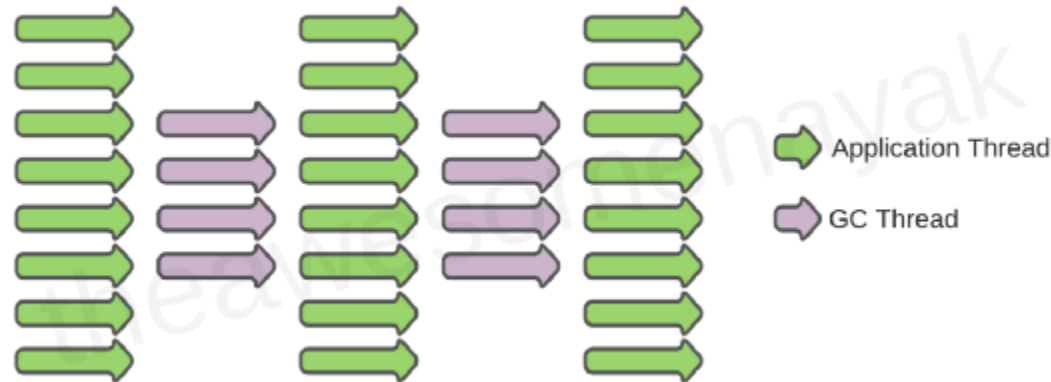
Запуск сборщика приводит к событию “остановки мира”, когда все приложение приостанавливает работу. Поскольку на время сборки мусора все приложение замораживается, не следует прибегать к такому в реальной жизни, если требуется, чтобы задержки были минимальными.

Аргумент JVM для использования последовательного сборщика мусора **-XX:+UseSerialGC**

Parallel GC

Параллельный сборщик мусора предназначен для приложений со средними и большими наборами данных, которые выполняются на многопроцессорном или многопоточном оборудовании. Это реализация GC по умолчанию, и она также известна как сборщик пропускной способности.

Несколько потоков предназначены для малой сборки мусора в молодом поколении. Единственный поток занят основной сборкой мусора в старшем поколении.



Parallel GC

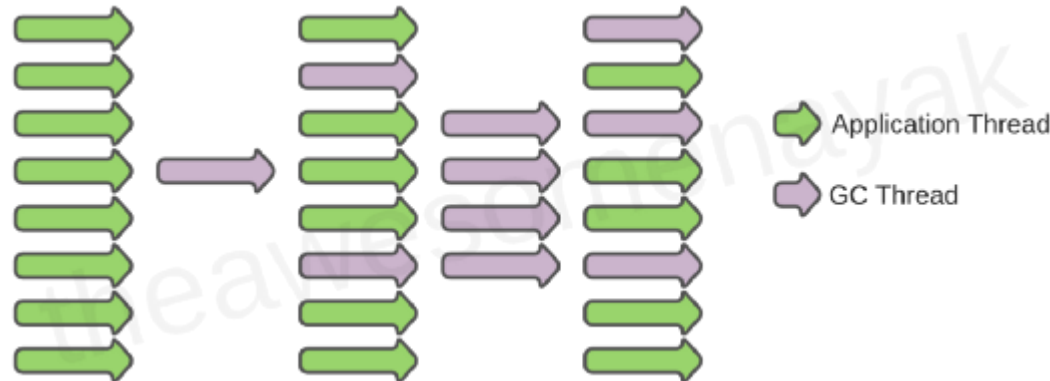
Запуск параллельного GC также вызывает “остановку мира”, и приложение зависает. Такое больше подходит для многопоточной среды, когда требуется завершить много задач и допустимы длительные паузы, например при выполнении пакетного задания.

Аргумент JVM для использования параллельного сборщика мусора: **-XX:+UseParallelGC**

CMS (Параллельная пометка и зачистка) GC

Concurrent Mark and Sweep Garbage Collector

Также известен как параллельный сборщик низких пауз. Для малой сборки мусора задействуются несколько потоков, и происходит это через такой же алгоритм, как в параллельном сборщике. Основная сборка мусора многопоточна, как и в старом параллельном GC, но CMS работает одновременно с процессами приложений, чтобы свести к минимуму события “остановки мира”.



CMS (Параллельная пометка и зачистка) GC

Из-за этого сборщик CMS потребляет больше ресурсов процессора, чем другие сборщики. Если у вас есть возможность выделить больше ЦП для повышения производительности, то CMS предпочтительнее, чем простой параллельный сборщик. В CMS GC не выполняется уплотнение.

Аргумент JVM для использования параллельного сборщика мусора с разверткой меток: **-XX:+UseConcMarkSweepGC**

G1 (Мусор — первым) GC

Garbage first Garbage Collector

G1GC был задуман как замена CMS и разрабатывался для многопоточных приложений, которые характеризуются крупным размером кучи (более 4 ГБ). Он параллелен и конкурентен, как CMS, но “под капотом” работает совершенно иначе, чем старые сборщики мусора.

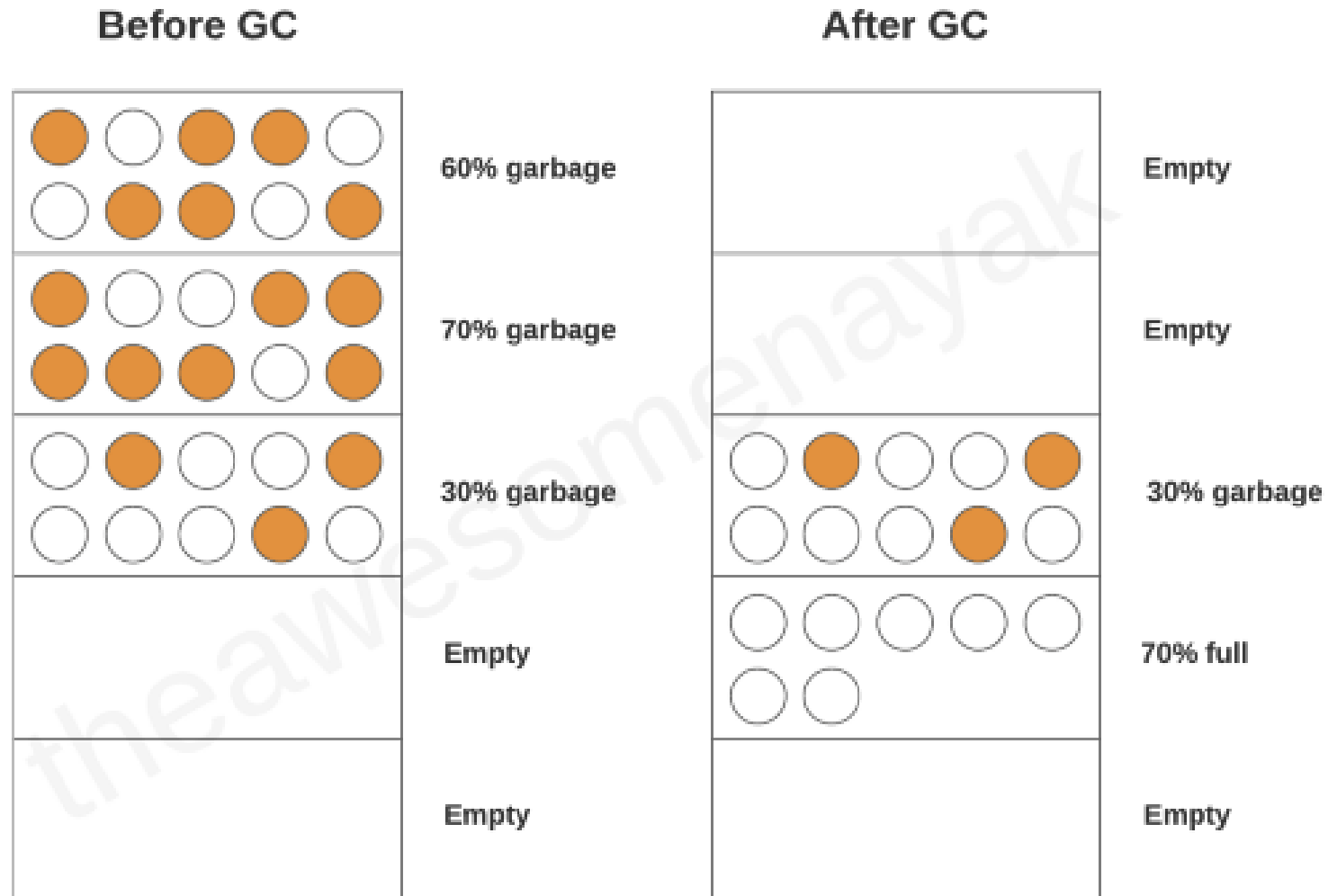
Хотя G1 также действует по принципу поколений, в нем нет отдельных пространств для молодого и старшего поколений. Вместо этого каждое поколение представляет собой набор областей, что позволяет гибко изменять размер молодого поколения.

G1 (Мусор — первым) GC

G1 разбивает кучу на набор областей одинакового размера (от 1 МБ до 32 МБ — в зависимости от размера кучи) и сканирует их в несколько потоков. Область во время выполнения программы может неоднократно становиться как старой, так и молодой.

После завершения этапа разметки G1 знает, в каких областях содержится больше всего мусора. Если пользователь заинтересован в минимизации пауз, G1 может выбрать только несколько областей. Если время паузы неважно для пользователя или предел этого времени установлен высокий, G1 пройдет по большему числу областей.

G1 (Мусор — первым) GC



G1 (Мусор — первым) GC

Помимо областей Эдема, Выживших и Старой памяти, в G1GC присутствуют еще два типа.

- Humongous (Огромная) — для объектов большого размера (более 50% размера кучи).
- Available (Доступная) — неиспользуемое или не выделенное пространство.

Аргумент JVM для использования сборщика мусора G1:
-XX:+UseG1GC

Epsilon GC

Epsilon — сборщик мусора, который был выпущен как часть JDK 11. Он обрабатывает выделение памяти, но не реализует никакого реального механизма восстановления памяти. Как только доступная куча исчерпана, JVM завершает работу 🤖

Его можно задействовать для приложений, чувствительных к сверхвысокой задержке, где разработчики точно знают объем памяти приложения или даже добиваются ситуации (почти) полной свободы от мусора. В противном случае пользоваться Epsilon GC не рекомендуется.

Epsilon GC

Если вдруг захотите поэкспериментировать и попробовать такой сборщик, то необходимо включить «экспериментальные» возможности JDK

Аргумент JVM для использования сборщика мусора Epsilon:

- XX:+UnlockExperimentalVMOptions**
- XX:+UseEpsilonGC**

Shenandoah GC

Ключевое преимущество Shenandoah перед G1 состоит в том, что большая часть цикла сборки мусора выполняется одновременно с потоками приложений. G1 может эвакуировать области кучи только тогда, когда приложение приостановлено, а Shenandoah перемещает объекты одновременно с приложением.

Shenandoah может компактировать живые объекты, очищать мусор и освобождать оперативную память почти сразу после обнаружения свободной памяти. Поскольку все это происходит одновременно, без приостановки работы приложения, то Shenandoah более интенсивно нагружает процессор.

Shenandoah GC

Данный сборщик тоже из разряда экспериментальных, поэтому
Аргумент JVM для сборщика мусора Шенандоа:

-XX:+UnlockExperimentalVMOptions

-XX:+UseShenandoahGC

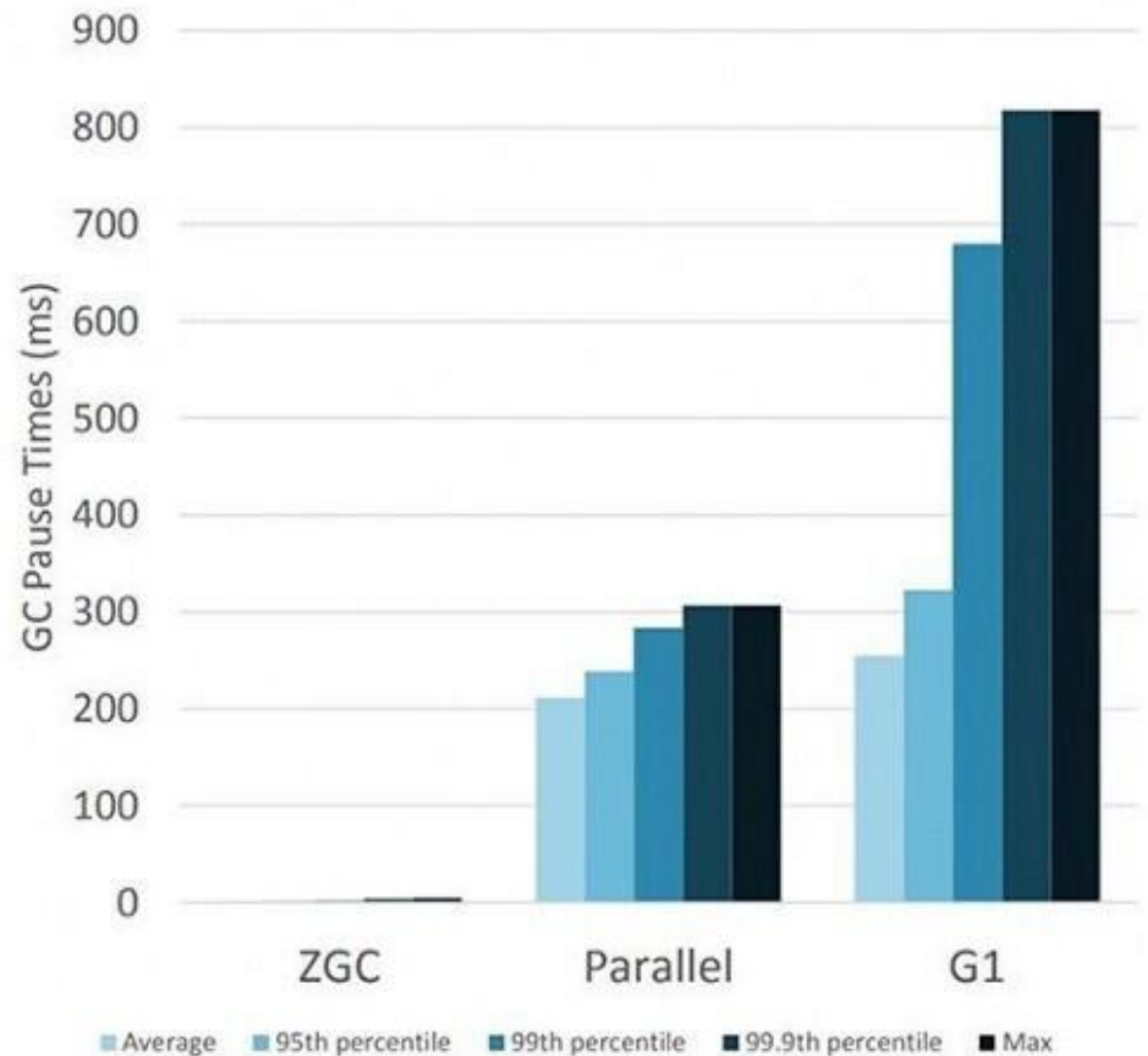
ZGC

Он предназначен для приложений, которые требуют низкой задержки (паузы в менее чем 10 мс) и/или задействуют очень большую кучу (несколько терабайт).

Основные цели ZGC — низкая задержка, масштабируемость и простота в применении. Для этого ZGC позволяет Java-приложению продолжать работу, пока выполняются все операции по сбору мусора. По умолчанию ZGC освобождает неиспользуемую память и возвращает ее в операционную систему.

ZGC

Таким образом, ZGC
привносит значительное
улучшение по сравнению
с другими
традиционными GCS,
обеспечивая
чрезвычайно низкое
время паузы



ZGC

Аргумент JVM для использования сборщика мусора ZGC:

-XX:+UnlockExperimentalVMOptions

-XX:+UseZGC

Примечание: Хотя и доступен на самом деле ещё в 12 версии JDK, но был официально релизнут только в 15-ой.

Не предназначен для использования на обычных персональных компьютерах, только для больших серверных решений с большим объёмом памяти !

Выбирайте сборщик правильно

Хороший ориентир в плане начальных настроек — характер настраиваемого приложения. К примеру, параллельный сборщик мусора эффективен, но часто вызывает события “остановки мира”, что делает его более подходящим для внутренней обработки, где допустимы длительные паузы.

С другой стороны, сборщик мусора CMS предназначен для минимизации задержек, а значит идеально подходит для веб-приложений, где важна скорость реагирования.

Заключение

Отдавайте предпочтение настройкам по умолчанию

Если у вас небольшое автономное Java-приложение, вам, скорее всего, не понадобится настраивать сборку мусора. И в целом это не совсем то, на что должен обращать внимание разработчик в своей повседневной жизни. Настройки по умолчанию отлично вам послужат.

p.s. Например в .Net реализация сборщика мусора едина